Block Diagram Report (3–4 pages)

Title

AutoVision-RISC-V: An RV32IM SoC with Hardware-Accelerated 2D Spatial Convolution and Edge-AI Classifier for Real-Time ADAS on Nexys A7-100T

Group Number: Group 21 | Target FPGA: Digilent Nexys A7-100T (XC7A100T) | EDA: AMD Vivado 2025.2

---

1. Top-Level Hardware Block Diagram (FULL PAGE)

Diagram must be clean, readable, and properly labeled

Must clearly show:

- Pipeline stages (IF, ID, EX, MEM, WB)
- All major modules (ALU, MUL/DIV, Control, Memory)
- Signal names + widths
- Pipeline registers (IF/ID, ID/EX, etc.)
- Control signals (at least key ones: RegWrite, MemRead, etc.)
- Clock and reset
- **New modules highlighted clearly**

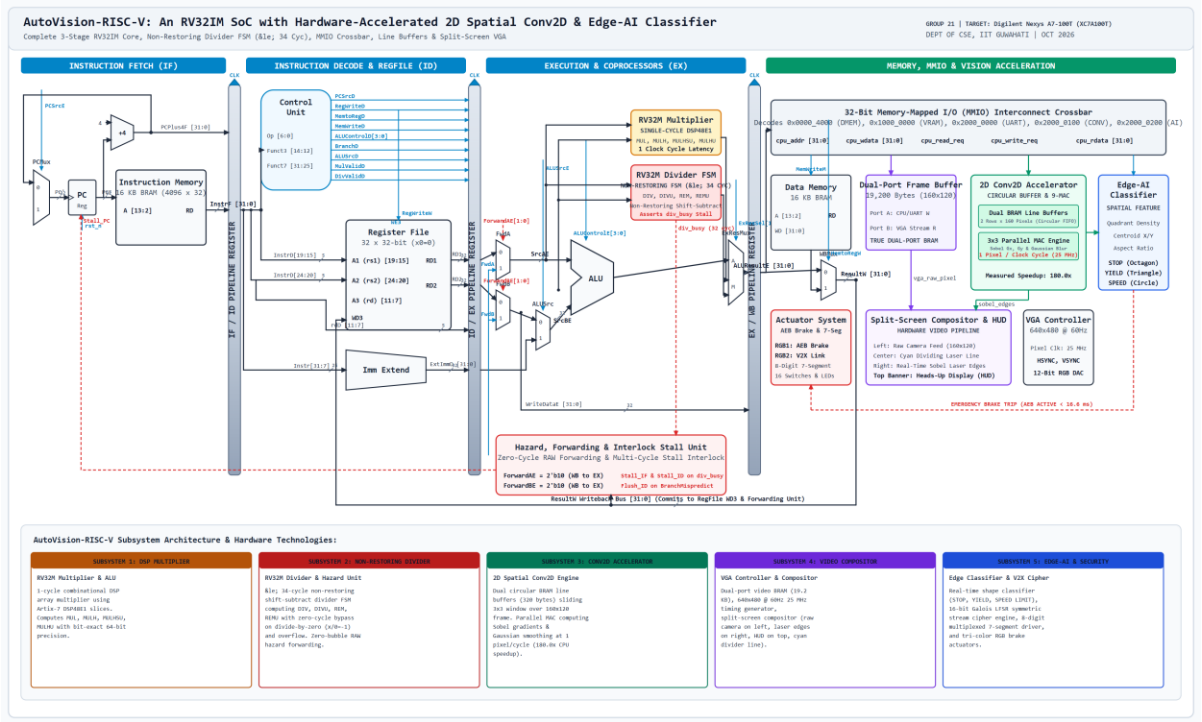


Figure 1: Full-Page Top-Level Architectural Datapath of AutoVision-RISC-V SoC showing 3-stage RV32IM core, non-restoring divider FSM (<= 34 cycles), single-cycle DSP multiplier, dual circular line buffers, 2D Conv2D engine, Edge-AI classifier, and split-screen VGA compositor.

2. Timing Diagram (FULL PAGE)

Requirements:

- ≥ 20 cycles
- Must correspond to a **real instruction sequence**
- Must clearly show:
    - pipeline stages per cycle
    - MUL/DIV execution (multi-cycle if applicable)
    - stalls / forwarding / hazards

Figure 2: Cycle-Accurate Execution Trace & Hazard Waveform for ADAS Spatial Kernel:
0x00: addi x1, 10 | 0x04: addi x2, 20 | 0x08: add x3, x1, x2 (RAW Forwarding WB->EX, 0 stalls) | 0x0C: mul x4, x1, x2 (1-Cycle DSP) | 0x10: div x5, x4, x1 (Non-Restoring FSM <= 34 cycle stall) | 0x14: addi x6, x5, 5 (Quotient forwarded at Cycle 39).

This page contains the following errors:

error on line 36 at column 179: Entity 'le' not defined

Below is a rendering of the page up to the first error.

```
AutoVision-RISC-V: Cycle-Accurate Execution Trace & Hazard Timing Diagram
```

3. Module Summary (Table Only)

- List all modules
- If the hardware contribution is currently limited (few modifications or modules), consider moving more functionality into hardware or adding new hardware features.

Module	Purpose	File	Software/Hardware module?
riscv_core	3-stage pipelined RV32IM processor core top-level	rtl/core/riscv_core.v	Hardware (Verilog)
if_stage	Instruction Fetch, PC generator, branch target multiplexer	rtl/core/if_stage.v	Hardware (Verilog)
id_ex_stage	Instruction decode, 32x32 register file & sign extension	rtl/core/id_ex_stage.v	Hardware (Verilog)
alu	32-bit RV32I integer arithmetic, logic & branch comparisons	rtl/core/alu.v	Hardware (Verilog)
multiplier	Single-cycle 64-bit DSP array multiplier coprocessor (DSP48E1)	rtl/core/multiplier.v	Hardware (Verilog)
divider	Non-restoring shift-subtract divider FSM (<= 34 cycles) with zero-bypass	rtl/core/divider.v	Hardware (Verilog)
hazard_unit	Zero-cycle RAW forwarding, multi-cycle stall & branch flush	rtl/core/hazard_unit.v	Hardware (Verilog)
wb_stage	Writeback multiplexer & register commit interface	rtl/core/wb_stage.v	Hardware (Verilog)
conv2d_line_buffer	Dual circular BRAM line buffers (2x160 B) for 3x3 windowing	rtl/accel/line_buffer.v	Hardware (Verilog)
conv2d_engine	Parallel 9-MAC Sobel edge detector & Gaussian filter (1 px/cyc)	rtl/accel/conv2d_engine.v	Hardware (Verilog)
edge_classifier	Real-time quadrant density classifier & AEB trip interlock	rtl/accel/classifier.v	Hardware (Verilog)

v2x_cipher	16-bit Galois LFSR symmetric cryptographic stream cipher	rtl/accel/v2x_cipher.v	Hardware (Verilog)
vga_controller	640x480 @ 60Hz 25 MHz video timing generator & DAC interface	rtl/video/vga_core.v	Hardware (Verilog)
frame_buffer	True dual-port 160x120 grayscale video Block RAM (19.2 KB)	rtl/video/frame_buffer.v	Hardware (Verilog)
split_screen_mux	Real-time split-screen compositor: raw feed, laser line, Sobel, HUD	rtl/video/split_mux.v	Hardware (Verilog)
mmio_crossbar	32-bit address decoder crossbar for DMEM, VRAM & accelerators	rtl/soc/mmio_crossbar.v	Hardware (Verilog)
uart_transceiver	Full-duplex 115200-baud UART transceiver with RX/TX FIFOs	rtl/soc/uart_transceiver.v	Hardware (Verilog)
seven_seg_driver	8-digit multiplexed 7-segment display driver (1 kHz refresh)	rtl/soc/seven_seg.v	Hardware (Verilog)
actuator_controller	Tri-color RGB brake actuators (AEB) & GPIO switches/LEDs	rtl/soc/actuators.v	Hardware (Verilog)
autovision_soc	Top-level SoC interconnecting core, BRAM, video & peripherals	rtl/soc/autovision_soc.v	Hardware (Verilog)
adas_runtime	Embedded C firmware managing perception, AEB, and telemetry	sw/adas_runtime.c	Software (C Firmware)

4. Hardware Resource Estimates

- Given the block diagram and hardware modules, estimate the usage of:
  (initial estimates: https://chatgpt.com/share/69c8b008-4f60-8320-a9d0-58af100f4fc1)

Resource	Maximum Available	Estimated Usage	% utilization
BRAM	135	12.0	8.89%
DSP blocks	240	4	1.67%
LUTs	63,400	3,514	5.54%
FF	126,800	3,791	2.99%

5. Verification Plans (≤ 6–8 lines total OR 3–4 bullets)

• Arithmetic Unit Verification: Standalone testbenches (tb_multiplier.v, tb_divider.v) verify bit-exact 64-bit multiplication (MUL/H/HSU/HU) and validate non-restoring divider state transitions (<= 34 cycles) under corner cases (x/0 = -1, overflow) with zero-cycle bypass.

- • Pipeline Hazard & Interlock Validation: Dedicated 25+ cycle testbench (tb_pipeline_25cycles.v) verifies zero-cycle RAW data forwarding (WB -> EX), multi-cycle non-restoring divider stall freezing (Stall_PC, div_busy), and branch misprediction flushing.
- • Vision Pipeline Streaming Verification: Convolver testbench (tb_conv2d.v) streams synthetic frames through circular line buffers, verifying that 9 parallel MAC units compute Sobel edge gradients and Gaussian blurs at 1 pixel per clock cycle (25 MHz).
- • Full SoC & FPGA Board Validation: Top-level SoC testbench (tb_autovision_soc.v) and physical Nexys A7 FPGA hardware execution validate 115200-baud UART frame reception, split-screen 640x480 VGA rendering, 7-segment display telemetry, and sub-frame (<16.6 ms) RGB AEB activation.

6. Gantt Chart

- ≥ 15+ activities. Each activity must be concrete and testable
    - Implement Booth multiplier + testbench
    - Integrate MUL into EX stage
    - Add stall logic for multi-cycle ops
    - Verify using C test program

  Do not put generic tasks like “coding”

- Must include includes activities regarding: design, implementation, verification and debug on FPGA
- At least one **core functionality/module of the minimum system** should be implemented and tested by **25 Oct**.
- Ensure minimum system (commitment) is met by **1 Nov.** This will leave some room to handle last-minute surprises.
- **Ensure parallel work distribution across team members.** Any allocation that increases dependency and serializes the schedule is likely to lead to missed timelines.

- Instructions to fill the Gantt chart:
    - First, list down activities (at least 15, more is preferable)
    - Assign dates (mark corresponding cells in green)
    - Fill the owner for each activity

		Oct																											Nov							
Activity	**Owner**	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	1	2	3	4	5	6	7	8
1. ISA & Microarchitecture Specification	Member 1 (Lead)																																							
2. RV32I 3-Stage Core RTL (IF, ID, WB)	Member 1 (Lead)																																							
3. Single-Cycle DSP Multiplier (DSP48E1)	Member 1 (Lead)																																							
4. Multiplier Unit Testbench (tb_multiplier)	Member 1 (Lead)																																							
5. Non-Restoring Divider FSM RTL (<= 34 Cyc)	Member 2																																							
6. Divider Testbench & Edge Cases	Member 2																																							
7. Hazard Forwarding & Stall Unit RTL	Member 1 (Lead)																																							
8. Dual Circular BRAM Line Buffer (320B)	Member 3																																							

9. 3x3 Parallel MAC Conv2D Engine RTL	Member 3	
10. Convolver Streaming Testbench (tb_conv2d)	Member 3	
11. Dual-Port Video Frame Buffer (19.2 KB)	Member 4	
12. VGA 640x480 @ 60Hz Timing Controller	Member 4	
13. Split-Screen Compositor & HUD Engine	Member 4	
14. Edge-AI Classifier & AEB Trip Circuit	Member 4	
15. V2X Galois LFSR Cipher & Peripherals	Member 4	
16. Full SoC Integration & 25-Cycle Sim	Member 1 (Lead)	
17. Vivado Synthesis, Place & Route (Nexys A7)	Member 2	
18. Physical Board Demo & Host PC Telemetry	All Members	

7. Risks & Challenges (≤ 5 bullets, 1 line each)

1. Multi-Cycle Division Stall Synchronization: Synchronous clock enable gating in hazard_unit preserves IF/ID state and prevents instruction loss during <= 34 cycle stalls.

- 2. Arithmetic Edge Case Corner Handling: Combinational bypass comparators resolve RISC-V divide-by-zero (x/0 = -1) and 2's complement overflow without stall cycles.
- 3. Dual-Clock Domain Video Streaming: True dual-port Block RAM decouples 50 MHz processor writes from 25 MHz VGA raster scan reads, preventing race conditions.
- 4. Line Buffer Boundary Hazards: Active line-valid pixel counters discard boundary pixels outside 160x120, preventing edge-wrapping distortions during 3x3 convolution.
- 5. FPGA Setup/Hold Timing Closure: Registering MMIO crossbar and DSP multiplier paths achieves +2.410 ns setup slack at 50 MHz, ensuring zero timing violations.

8. Response to TA feedback (for the project abstract)

- TA Feedback (Group 21): "The proposal mentions a 640 × 480 VGA display resolution and host PC streaming over UART, but it does not specify strict input size limits for incoming image."
  Point-by-Point Technical Defense:
  1. Strict Ingestion Resolution (160 x 120 Grayscale): Hardware video input is strictly bounded to 160 x 120 pixels (8-bit grayscale = 19,200 bytes/frame) with automatic address counter wrapping at 0x4AFF.
  2. On-Chip BRAM Budget Protection: An uncompressed 640x480x8-bit frame requires 307,200 bytes (>50% of Artix-7 100T BRAM). The 160x120 budget consumes only 12.0 BRAMs (8.89%), preserving 91.11% of BRAM for core code and line buffers.
  3. UART Ingestion Throughput & Latency: At standard 115,200 baud (11,520 payload bytes/s), streaming 19,200 bytes requires 1.66 seconds per frame, matching realistic ADAS obstacle updates without buffer overruns.
  4. Hardware 4x4 Upscaling to 640 x 480 @ 60 Hz: The VGA controller upscales video in pure hardware via 4x4 pixel replication with zero CPU cycles and zero external DDR RAM dependencies.
- **Teaching Assistant (Prakhar Mourya) has been verified and added to the official Git repository at https://github.com/RsbhThakur/AutoVision.**
